kid_buddy — 会讲故事的儿童 AI 伙伴

2026 首届 openvela AI 硬件开发者大赛 · 参赛作品介绍

作品名称	kid_buddy
所属赛道	AI 硬件产品创新
队伍编号	086
硬件平台	Gemini-S1（全志 R528 / sun8iw20，128M NAND，单麦 + 喇叭 + LCD）
专属仓	https://github.com/open-vela/contest2026_086_ditiekouyegoulianmeng
应用版本	v2.70

一、用户故事

一块巴掌大的板子，加一个能听会说的儿童玩伴。

孩子对着它喊一声「你好，openvela」，它醒过来。之后可以随便聊天问问题，也可以**挑一个 角色进入一段冒险故事**——老师、故事家、科学家、朋友，每个角色的说话方式和故事线都不 一样。故事是**有分支**的：孩子选「推开木门」还是「绕到屋后」，后面走的是两条路。

屏幕上是一张脸。 界面没有页面、没有按钮、没有菜单，整块屏就是一张用圆和弧线画的 脸：平静、高兴、惊讶、专注、难过、困倦六种表情，配上头顶的小道具（转圈的三点、随说话 音量起伏的弧、摇晃的铃铛、飘的泡泡），把二十来个设备状态翻译成孩子一眼能看懂的表情。 屏的右缘露出四条彩色书签带，点一下换角色。底部三分之一是一块三行的字幕板：第一行是 孩子刚说的话，下面两行是它的回复正文——**正文放不下会自动往上滚**，每两秒半走一行， 走到底再回到开头，所以整段回复都读得到。

字幕是后来加回来的——最初想让屏幕上一个字都没有，看完脸的表情出图才改的主意。原因 不是孩子（六岁的孩子读不了那段正文），是**给旁边的大人看的**：这块板子的语音全在 云端，网络一不好就会哑，而**哑掉听起来和「玩具不想理你」一模一样**。屏幕上有字，是 唯一能证明它还活着的东西。所以语音一旦失败，脸一定会显「难过」——字幕是旁证，不是提示。

跟别的语音助手相比，它有三个地方是刻意做的：

故事和闲聊不混在一起。 孩子中途问一句「恐龙为什么会灭绝」，不会把剧情带跑偏。 两个会话在系统里是分开存的，回来接着讲还在原来那条线上。

孩子走开了会自己接上。 讲了一半人跑了，过一会儿它会主动问：「上次的冒险讲到一半 啦，还要继续吗？」

回答完还给一次机会。 回复完的十几秒内，孩子直接说「第二个」或者「继续」就行， 不用再喊一遍唤醒词。小孩说不出「请重复上一句」这种话，这个窗口是替他们省事的。

另外，家长交代的提醒到点播报之后，孩子回一句「知道了」「好」「做完了」，会通过 MQTT 推一条回执到家长端——不是监控，是让家长知道交代的事确实传到了。

孩子玩过的冒险也留得下来。 家长在手机上点一下「同步」，孩子在故事里做过哪些选择、 听的是什么情节，就整段取到手机里存着，按时间分成一段段冒险。**这份档案存在家长手机本地**， 板子重启、刷新都不影响——孩子讲过的故事，家长翻得到。

功能清单

功能	怎么触发	说明
语音唤醒	说「你好，openvela」	板子没有本地 KWS，唤醒是在 ASR 转写结果上做归一化字符串匹配，认大小写/空格/标点差异和几个音译写法
自由问答	唤醒后直接说	ASR → LLM → TTS 全链路，流式按句播报，首句出声不用等整段生成完
四个角色	点屏右缘的四条书签带	老师 / 故事家 / 科学家 / 朋友，各有说话方式与知识范围，换角色会换背景色
分支剧情故事	说「我们来玩冒险游戏」	讲背景 → 给 2~3 个选项 → 孩子语音选 → 剧情按选择走 → 再给选项，直到结局
剧情与闲聊隔离	自动	故事模式和自由问答各存一份会话历史，中途问别的不会把剧情带跑偏
断点续讲	自动	故事进度落盘，讲一半断电，下次开机接着上次那条线讲
主动续讲	孩子走开后	故事讲一半没人接，过一会儿 Agent 主动开口问「上次的冒险讲到一半啦，还要继续吗」
追问窗口	回复后十几秒内直接说	不用再喊唤醒词，说「第二个」「继续」就行——小孩说不出「请重复上一句」这种话
定时提醒	说「提醒我 10 分钟后喝水」	模型自己调 cron 工具建定时任务，

		到点铃铛响 + 出声，需要孩子确认才消
家长端确认回执	提醒播报后孩子说「知道了」「好」「做完了」	通过 MQTT 推一条回执到家长端网页，让家长知道交代的事传到了。只认短答句，孩子要是接着说「知道了，但我不想喝」，那句话会照常走正常问答
孩子的冒险记录	家长端点「同步」（连上后也会自动同步一次）	把孩子角色扮演会话的文本记录取到手机上，按时间切成一段段冒险展示——孩子说的每句话标蓝「孩子：」，故事正文缩进灰字。**长期档案存在手机本地**（板子只留最近 20 条并滚动截断，每次同步往手机上合并去重），所以孩子玩过的故事不会因为板子刷新而消失
全屏脸谱界面	自动	纯 LVGL 图元画的六张表情脸 + 六种道具，把二十来个设备状态翻译成表情，屏上一个字都没有
字幕板 + 滚动	自动	底部三分之一：孩子的话一行、回复正文两行，正文放不下会自己往上滚一圈
失败可见	这一轮请求失败时	显「难过」5 秒并播一句中文安抚语——语音哑掉必须和「玩具不想理你」区分开。（断网待机是另一张脸：**困倦 + 泡泡**，因为「帮不上忙」和「我做错了事」对孩子不是一个意思）
自愈	自动	开机先校时（板子无 RTC，时钟停在 1970 会让 TLS 全挂）、回复卡死有看门狗恢复、TLS 握手失败逐 IP 重试

二、怎么跑起来

方式一：直接烧预编译固件（推荐）

```
image/kid_buddy_gemini-s1_uart0_128Mnand_v2.70.img
```

用全志 PhoenixSuit / LiveSuit 按常规流程烧进 NAND。上电后启动脚本会自动拉起图形界面 和语音后台，不需要再编任何东西。

确认镜像生效：开机时串口（uart0，115200 8N1）会打出当前应用版本号和实际屏幕分辨率：

```
[kid_buddy] GUI v2.70 display 320x240
[kid_buddy] face UI v2.70 on 320x240 (face band 162, panel 6,166 278x69, 33 cols)
```

屏幕本身不再显示版本号——右上角那个位置现在是书签带，整屏只有一张脸和底部三行字幕。

方式二：从源码编

本仓自带完整的板级配置（`board/nsh_minidisplay/`），不需要另外改 `vendor` 树：

```
repo init -u https://github.com/open-vela/contest2026_086_ditiekouyegoulianmeng \
  -b dev-ai-contest-2026 -m contest2026_086_ditiekouyegoulianmeng.xml
repo sync -c -j8

# 在工作区根目录
./build.sh contest2026_086_ditiekouyegoulianmeng/board/nsh_minidisplay
```

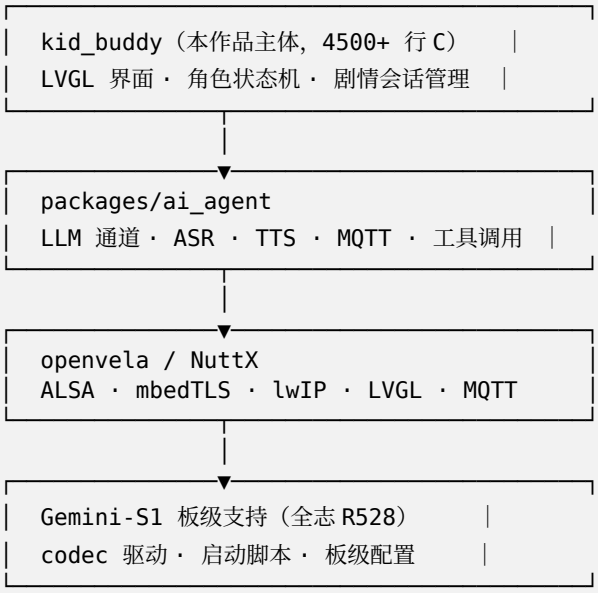
编译完成后打包：

```
cd vendor/allwinnertech/lichee
./tools/scripts/pack_img.sh -c sun8iw20p1 -p rtos -b r528s3-gemini-s1 -o nuttx \
  -d uart0 -s none -m normal -w none -v none -i none -t "$PWD" \
  -f r528s3/gemini-s1_nand -g r528s3/gemini-s1_nand
```

> ***注意：这套构建不跟踪头文件依赖。*** *改过 `packages/ai_agent` 的头文件之后，相关的目标文件不会自动重编，留着旧的会出现结构体大小对不上的问题（症状是读出 240 这样的垃圾值，而不是编译报错——很难往这上面想）。*
>
> *正确做法是**把那个模块的目标文件删掉再编**：*
>
> ```
> bash find packages/ai_agent -name '*.o' -delete make -C nuttx -j2
> ```
>
> ***不要用 `make -C nuttx distclean`。*** *它会重下第三方源码（`quickjs` 的补丁在重下之后对不上，直接断编译），还可能清掉 vendor 侧的 `.built` 标记，导致归档里缺对象、链接时报一批 `*_main` 未定义。`make clean` 又清不到位。删 `.o` 是这里最小的、精确的做法。*
>
> *另外**不要删 `apps/libapps.a`** —— 删了会遇到同样的问题：一批 `*_main` 未定义，看起来像 registry 脏了，其实是归档缺件。*

三、技术方案

3.1 整体结构



3.2 语音链路

采集：本地能量阈值 VAD 掐出一句话（20ms 一帧，起止阈值带迟滞），只在没有回复在播 的时候才开麦克风。

识别：音频送 MiMo ASR 转写。

唤醒：板子上没有本地 KWS 引擎，唤醒是在**转写结果上做归一化匹配**——大小写折叠、 空格标点当分隔符、并认几个音译写法。这样「你好，openvela」「你好 openvela」 「Hello，OpenVela」都能唤醒。

合成：MiMo TTS，**一次回复只开关一次音频设备**，按句流水线预取。

3.3 剧情会话隔离

这是这个作品里最能体现「不是套壳」的一块。

自由聊天和冒险故事如果共用一份对话历史，剧情状态会被闲聊污染（反过来也一样：家长问 一句天气，故事里的角色会突然开始聊天气）。

做法是：进入故事模式时，这一路的请求带一个**独立的会话标识**，`ai_agent` 的会话管理 器就把它的历史单独存一份。进出故事模式由语音触发（「我要听故事」/「不玩了」）， 状态落盘，中途断电再开机还能接着上次的剧情。

3.4 角色的人设从哪来

`app/kid_buddy/skills/` 下有 5 份英文 Markdown（`role-teacher.md` / `role-storyteller.md` / `role-scientist.md` / `role-friend.md` / `role-switch.md`），是人设的**设计稿**：性格、 知识范围、说话方式、安全边界、示例对话。

运行时并不读这些文件。 真正注入提示词的是 `kid_buddy.c` 里 `g_roles[]` 的 `system_prompt` 中文字符串。它是在英文设计稿的基础上改写的，并加了几条面向这个硬件的 硬约束——只讲简体中文、回答 100 字以内、禁止 Markdown 标记、禁止任何引号（TTS 会把引号 念出来）、以及"孩子想玩故事时切换成故事主持人"。

代价说清楚：**换角色、改人设要改 `g_roles[]` 并重新编译**，不加载文件。 保留英文稿是因为它比压缩后的提示词完整得多，是改人设时的参照；两份是手工同步的。

3.5 界面：一张纯几何的脸

整块屏（320×240）是一张用 LVGL 图元画的脸，**不带任何美术素材**——没有 PNG、没有字体 图标，全部是圆角矩形、圆和弧：

部位	画法	怎么动
背景	屏幕对象本身	换角色时换色
眼白	圆角矩形（半径固定在眼宽的 37%）	只改高度 = 眨眼 / 眯眼
瞳孔 + 高光	小圆	改坐标 = 视线漂移
眉毛	细长圆角矩形	绕内端旋转 = 抬 / 压 / 皱眉
嘴	`lv_arc`	改起止角 = 笑 / 平 / 撇嘴
道具	三点 / 弧 / 铃铛 / 泡泡 / 弯钩 / 小手	转圈 / 随音量起伏 / 摆动 / 上浮
书签带 ×4	贴右缘的圆角矩形	选中态抽出来 + 提亮
字幕板	一块半透明底板 + 3 个标签	第 1 行孩子的话；下方是一个两行高的窗口，回复正文（最多 8 行）在窗口后面向上滚动

一张 100 ms 的定时器驱动全部表情变化，并且**只写真正变了的属性**。这一条不是优化技巧， 是这个板上必须守的规矩。

3.6 用到了 ai_agent 的哪些能力

这是本作品和「自己接一个大模型写个语音助手」最大的区别：**判断和动作都在框架里，不在应用里**。

ai_agent 的能力	我们怎么用的

主动任务机制	提醒是模型自己调 cron 工具建的定时任务（`infra/cron_service.c` 落到 `/data/ai_agent/cron.json`，重启不丢）；故事讲一半孩子走开了，由 idle 触发主动续讲
意图分流	「提醒我十分钟后喝水」没有被应用层的关键词表分发，是技能摘要（`skill_loader_build_summary()`）进系统提示词后，模型自己决定去调 cron 工具建任务。（反过来，故事模式的进出是应用层关键词表判的——`chat_id` 必须在请求发出去之前就定下来，模型没办法事后把一轮对话挪进另一个会话）
Tool 调用	Agent 主循环带着工具表跑（`agent_loop.c` 挂 `tool_registry_get_tools_json()`，工具结果回灌后再让模型说下一轮）。提醒的增删查走 cron 工具，讲故事走读文件
自然语言 → 结构化	「十分钟后喝水」里的时间和事由由模型解析成 cron 的结构化参数，不是我们写死的解析器
Skill 体系	自定义技能 `story-rpg`：一份 Markdown 放在 `/data/agent/skills/story-rpg.md`，开机由 `skill_loader_init()` 写入，内容是「怎么起头、每轮怎么给选项、选项要用完整口语不能念 ABC、怎么续、怎么收尾」的操作手册。模型读了它才会讲故事
会话管理	故事模式走独立 `chat_id`，历史与自由问答各存一份
记忆	历史落盘 + 开机读回，所以断电再开机能接着上次的剧情讲
多渠道	同一套 Agent 逻辑同时接语音（板载麦 / 喇叭）、CLI（NSH）、MQTT（家长端网页），我们只在渠道特性上做适配

板子这一侧，`kid_buddy.c` 通过 `velaclaw_ask()` 把请求推进消息总线，剩下的都在框架里跑。

四、做这个作品踩过的坑（也是它值钱的地方）

这部分写在作品介绍里，是因为它代表了真实的工程量。每一条都是先误判、再定位，最后改到 驱动层、协议层或绘制层才解决的。

现象	真正的原因	层次
只有第一句回复有声音，追问全哑	codec 每次打开都无条件重配 DAC 时钟，把时钟打毛；且	内核驱动

	`dapm_state` 没重置，模拟通路不再重跑	
播放时麦克风报 `request dma chan failed`	单麦板子上采集与播放共享同一条 codec DMA，忙标志置晚了	驱动 + 应用
老是自己唤醒自己	单麦板子用了 Kconfig 默认的采集增益 6，底噪被放大 36 倍顶穿 VAD	配置
TTS 时有时无	云端端点是个负载均衡，个别后端 IP 在某段时间握手必失败	网络协议
每句话卡二十秒	TLS 客户端对 keep-alive 的 chunked 响应一直读到服务端关连接，而 keep-alive 就是不会关	协议实现
提醒到点不响	单线程的消息分发被一个卡住的 MQTT 连接阻塞，整条队列出不去	并发
多轮对话返回上一次的答案	回复缓存的键用了被截断的当前消息，前缀相同时撞键	状态管理
结构体字段读出 240 这样的垃圾值	构建系统不跟踪头文件依赖，改过头文件后链接的是旧目标文件	构建系统
改过启动脚本、重编也报成功，行为却没变	构建系统只把 `etc/init.d/rcS` 列为依赖，而它全部内容是 `#include "rcS.nsh"`，`rcS.nsh` 压根不在依赖表里，改它永远不触发重生成	构建系统
烧录后白屏，只有右下角一个橙色长方形	板级配置里 `CONFIG_LCD_FRAMEBUFFER=y` 会让驱动的 `#if/#elif` 走 framebuffer 分支，`lcddev_register()` 成了死代码，`/dev/lcd0` 压根没注册出来	板级配置
配好网络和 APIKEY 之后毫无反应	板子没有 RTC，系统时钟停在 1970，HTTPS 证书的 notBefore 校验全部失败	系统 / 网络协议
待机时板子也一直在刷屏	LVGL 在这块板上是**整屏单缓冲**（`CONFIG_LV_NUTTX_LCD_BUFFER_COUNT=1`），任何一处失效都要重绘并刷**整块屏**，一次 153,600 字节、"只重绘脸那块"的优化完全不成立	图形栈
回复念到一半就没了，脸上也没有	LVGL 标签**会裁掉自己的溢出**。换	应用逻辑

省略号	行既发生在格子用完时、也发生在 `\n` 和空格处，只按格子数算行数会多算出一行，多出来的那行不报错、不画出来、安静消失	
算好的行宽比实际窄 8px，于是又折了一行	判断全角用的「U+2000 以上是宽的」是错的：实测字体里有 **128 个更小的码点也是全角**（`° ± × ÷`、希腊字母、西里尔字母），有一个「°」算成半角就正好够再折一次行	字体度量
说话时偶有爆音	屏走的 SPI1 和音频采集/播放共用 DMA 通道，动画帧率一高就抢	硬件
正文两行装不下，后面的内容够不着	提示词要求模型答 100 字，两行 33 格只有 66 格——**上限是屏幕给的，不是模型给的**。解法是把上面那条"标签会裁掉自己的溢出"反过来用：标签比窗口高、放进一个正好两行高的对象里，定时往上移，同一套裁切就从限制变成了滚动的窗口	应用逻辑

后面这几条是这个作品的两块新界面带来的：前几条是重做界面时**踩进去**的，共同点是**从代码 上完全看不出来**——要把 LVGL 的源码读到绘制层，或者把字体文件解析出来量一遍，才知道真正 的原因在哪；倒数第二条（SPI 和音频抢 DMA）不是撞上的，是**在驱动源码的注释里先读到、 再主动绕开**的，它决定了整套动画的帧率预算怎么分；最后一条是把同一个裁切机制反过来用 ——先当成坑踩了一次，再当成工具用了一次。

这些改动里，与 openvela 公共仓有关的部分都按大赛规则单独提了 PR，没有直接改本地副本。

五、AI 开发流程

全程使用 Claude Code。 从 7 月 25 日到 9 月 20 日，近两个月的开发周期。应用本身、 界面重做、上游那 5 个 PR、以及第四节里全部排障，都是在这个流程里做出来的。

完整对话记录在 `logs/` 目录里：**32 个有产出的日子、17,680 条事件**，其中 11,340 条是 工具调用（能看到改了哪些文件、跑了哪些命令）。整个开发过程在一个会话里连续完成， 所以只有一个 `.jsonl`，没有分段或筛选；`manifest.json` 记录事件总数与会话清单。

AI 在这个项目里主要不是「写代码」，而是**在信息不足的时候帮忙缩小范围**：

- 上面那张表里的「真正的原因」，大部分是先由 AI 根据日志和代码提出假设、再去验证的。 比如「TTS 时有时无」一开始怀疑是证书算法不被支持，去查那个 IP 才发现证书是健康的， 真正的问题是负载均衡的坏窗口。
- 构建系统的坑（陈旧目标文件、配置不同步）也是 AI 从「代码明明改了但行为没变」这个 现象反推出来的。

沉淀的技能。 我们没有重写官方技能集已经覆盖的东西（编译、驱动、音频质量分析、 提 PR 流程），而是把官方没覆盖、我们踩出来的部分写成了 4 个 Skill，放在本仓 `.claude/skills/`：

Skill	内容
`openvela-build-traps`	陈旧目标文件、`clean` 与 `distclean` 的区别、`build.sh` 会覆盖板级配置、第三方 app 补丁失效
`ai-agent-board-bringup`	ai_agent 的配置矩阵、按 IP 回退的 TLS 重试、chunked 响应读不完、MQTT 双线程互踢与队列阻塞
`voice-loop-tuning`	单麦增益、DMA 争用、codec 二次打开无声、唤醒词归一化匹配
`openvela-contest-submit`	ruleset 与 branch protection 的区别、CLA 邮箱、只能 rebase 合并导致 SHA 重写

每个 Skill 的 `description` 里都写了触发条件（`Use when: ...`），Claude Code 会在遇到 对应症状时自动选用。

六、已知限制

写在前面，不藏：

1. **语音链路全在云端**。 LLM、ASR、TTS 都走网络，断网时只剩图形界面可用。
2. **唤醒不是本地声学唤醒**。 板子上没有 KWS 引擎，实际是 VAD + 云端 ASR 转写后做 字符串匹配，所以环境吵的时候得离近一点说。唤醒词的匹配写法做过离线单元测试 （含正例和不该唤醒的负例），但**没有针对真实 ASR 输出做过大规模校准**——日志里会 打印原始转写，遇到没覆盖的写法加一条即可。
3. **上游 PR 尚未合入**。 作品对 `packages_ai_agent` 和 `vendor_allwinnertech` 的改动 已按规则提了 PR，但组委会的评审有 CODEOWNERS 闸门，参赛者无法自行合并。**这不影响 交付**：改动过的 43 个文件的成品放在本仓 `upstream/` 下，manifest 在 `repo sync` 时覆盖回工作区，所以「直接烧 `image/` 里的固件」和「从源码编」两条路都完整可用。
4. **预编译固件的验证程度较低**。 这条单独放在下面 6.1 说清楚。

5. 服务端（LLM / ASR / TTS / MQTT broker）需要自行搭建，作品本身不含服务端。

6.1 交付镜像的验证程度

上板验证过的：**v2.67**（2026-09-19 逐条跑完）。决赛版 **v2.70** 在它之上增加了四项 现场可靠性改动——开机连多个候选网络（手机热点兜底）、开机自检写字幕板、MQTT 出站看门狗、 输出侧内容安全过滤（详见技术报告 3.4.6）——以及一项新功能：**孩子的冒险记录** （家长端把孩子的角色扮演文本历史取到手机本地长期保存，见技术报告 3.2.3 第 7 项）。 这些改动都以「不改动主链路」为设计约束，各自的验证方法逐项列在仓内 `docs/上板验证清单.md`，随决赛演示一并复验。

v2.67 当时跑过的：脸谱界面——脸、 字幕板、书签带、六个表情、全部道具、触摸、以及两条失败路径（这一轮请求失败 → 「难过 + 中文安抚语」；断网待机 → 「困倦 + 泡泡」）——连同这一版新增的**字幕滚动**（正文标签允许长到 8 行，前面开一个两行高的窗口， 每 2.5 秒上行一行、到底回到开头）一起在真机上点亮并逐条跑过，同批的回归项 （唤醒、追问窗口、提醒到点、故事模式分叉、MQTT 上报）也一起过了。再往前，v2.55 上板复烧确认了白屏修复（`/dev/lcd0` 未注册，见第四节），截至 v2.53 的各功能版本 （唤醒、ASR/LLM/TTS、提醒、RPG 会话隔离、MQTT 上报）都在真机上跑过， MQTT + 角色扮演回路是 2026-09-09 上板验证通过的。

还留着一个没大规模校准的地方：唤醒词的匹配写法只做过离线单测，没有对着真实 ASR 输出成规模地校准过——喊不醒的时候，日志里 `wake: heard "..."` 会打出原始转写，照着 往里加一条音译即可。

镜像的构造校验（在 **v2.70** 交付镜像上实测，结果是确定的，作为交叉验证）： 版本号字符串在镜像里**只出现一次**且为 `v2.70`（`v2.67` / `v2.68` / `v2.69` 均为 0 次）； `date -s` 一次都不出现（时钟自愈已挪进应用，见第四节）；唤醒词 「你好 openvela / 欧本维拉」都在，且唤醒别名表里没有旧唤醒词；内建程序名 `kid_buddy`、 两条开机日志格式串（`GUI %s display %dx%d` 与 `face UI %s on %dx%d`）、 决赛版新加的自检格式串（`selfcheck: net=%d clock=%d`）、WiFi 候选目录名（`networks.d`）、MQTT 看门狗标记（`WATCHDOG`）、以及冒险记录的协议标记 （`story_log`）都能在二进制里查到； `.sha256` 与镜像一致。

真要有问题，v2.60 ~ v2.69 十份历史镜像在我们本地留着（每份 27 MB，没有进仓）， 可以直接烧回去。其中 `v2.68`（只含前两项改动）是内容安全过滤的**回退点**—— 万一过滤出现误杀，烧回它即可去掉这一层。

七、目录说明

```
app/kid_buddy/            作品主体：4500+ 行 C + 5 份角色人设设计稿
board/nsh_minidisplay/    板级配置（defconfig + Make.defs），自带即可编译
image/                    预编译整机固件 + SHA256
logs/                     AI Coding 对话日志（2026-07-25 ~ 09-20）
.claude/skills/           本队沉淀的 4 个开发 Skill
docs/                     作品介绍、演示脚本、上游 PR 备忘
contest2026_086_ditiekouyegoulianmeng.xml
                          manifest，把 app/kid_buddy 软链进编译树
```